

# MCSAT Modulo Transcendental Arithmetics

Jorge Gallego-Hernández   
 IMDEA Software Institute  
 Universidad Politécnica de Madrid  
 Madrid, Spain

Enrico Lipparini   
 University of Cagliari  
 Cagliari, Italy

Alessio Mansutti   
 IMDEA Software Institute  
 Madrid, Spain



**Abstract**—We propose a framework for solving quantifier-free formulas from (undecidable) extensions of non-linear real arithmetic (NRA) with transcendental functions, such as exponential and trigonometric ones. The framework extends the Model Constructing Satisfiability calculus (MCSAT), and leverages procedures for NRA and methods from real analysis.

At its core, our procedure abstracts the input formula to NRA, and lets MCSAT and an NRA plugin incrementally build a partial model of the abstracted formula. A *Transcendental Real Arithmetic plugin*, acting as an intermediary between MCSAT and the NRA plugin, ensures the consistency of the partial model and is responsible for refining the abstracted formula.

We implemented our procedure in the Yices2 SMT solver for the sine and exponential functions, and conducted an extensive empirical evaluation that shows that our prototype outperforms state-of-the-art solvers on both SAT and UNSAT instances.

## I. INTRODUCTION

Many industrial applications in computer science and engineering reduce to deciding formulas over the real numbers that involve transcendental functions such as the sine and the exponential [1], [2]. Yet, deciding such formulas is notoriously difficult, and in many cases outright impossible:

- The problem is already undecidable for univariate formulas with addition, multiplication, and the sine function [3]; and only known to be decidable subject to Schanuel’s conjecture for the exponential [4].
- Even representing and checking solutions is sometimes not possible, in particular for formulas whose only solutions are transcendental numbers, i.e., numbers that are not roots of any polynomial with integer coefficients.

These facts motivate the design of principled but incomplete methods to tackle problems involving transcendental functions on a best-effort basis. In this paper, we present such a method.

Specifically, we address the Satisfiability Modulo Theories (SMT) problem for Transcendental Real Arithmetics (TRAs), that is, deciding the satisfiability of quantifier-free formulas from first-order theories extending Non-linear Real Arithmetic (NRA) with transcendental functions. Working in the Model Constructing Satisfiability (MCSAT) framework (which generalizes ideas from CDCL to the theory level), we design a procedure applicable to any such theory whose transcendental functions are total and interval-computable.

At the high level, our procedure abstracts an input TRA formula to NRA, and uses NRA lemmas to incrementally refine the abstraction. A *Transcendental Real Arithmetic plugin* (TRA plugin) acts as an intermediary between the core solver

of MCSAT and a (already existing) plugin for NRA. The role of the TRA plugin is to ensure that the trail of MCSAT is “consistent” in the following sense: for every literal fully assigned in the trail, its truth value in the NRA abstraction matches the truth value in the TRA concretization. When the TRA plugin detects an inconsistency, it triggers a conflict, producing a valid NRA lemma that is conjoined to the NRA abstraction, effectively refining it.

In TRAs, evaluating the truth value of a fully-assigned literal is in general an undecidable problem. We adopt the practical strategy of relaxing the consistency check of the TRA plugin up to a tolerance  $\delta > 0$ . This means that, in case it is not possible to disprove consistency using over-approximations of the transcendental functions that depend on  $\delta$ , the TRA plugin considers the check passed, and marks the literal as  $\delta$ -consistent. The benefit of this approach is that  $\delta$ -consistency is decidable for interval computable functions, and hence the TRA plugin does not get stuck during its consistency check.

The procedure can operate in two modes:

- 1) With tolerance  $\delta$  fixed throughout the procedure and provided by the user. In this mode, the procedure may return SAT, UNSAT or  $\delta$ -SAT. The first two cases carry their usual meaning. In the case of  $\delta$ -SAT, the procedure has found a solution to the NRA abstraction for which all literals were proven consistent or  $\delta$ -consistent by the TRA plugin, with at least one being  $\delta$ -consistent.
- 2) With tolerance not fixed, and the procedure may only return SAT or UNSAT. A tolerance  $\delta$  is still used internally, but rather than answering  $\delta$ -SAT, the procedure decreases  $\delta$  and backtracks to the empty trail, while preserving all NRA lemmas produced so far.

While our procedure works for any TRA whose transcendental functions are total and interval computable, we further devote special attention to the case of the sine and exponential functions (plus the constant  $\pi$ ), providing more precise NRA lemmas based on Taylor and Padé approximations.

For sin, exp and  $\pi$ , we implemented a prototype<sup>1</sup> of our procedure within the Yices2 SMT solver [5]. We compared the prototype to MathSAT [6], cvc5 [7] and dReal [8] on the 2512 instances from [6]. Our experimental evaluation shows that our tool outperforms other tools on both SAT and UNSAT instances (and, when  $\delta$  is fixed, also on  $\delta$ -SAT instances).

<sup>1</sup>Available at [github.com/arith-lab/yices-tra](https://github.com/arith-lab/yices-tra).

## II. PRELIMINARIES

In this section we introduce the necessary background material on arithmetic theories over the reals, and provide a high-level overview of the MCSAT framework. We assume familiarity with propositional satisfiability and the Conflict-Driven Clause Learning (CDCL) algorithm (a brief overview of CDCL is given in the appendix).

We recall that the *Satisfiability Modulo Theory (SMT) problem* asks for the satisfiability of a quantifier-free formula in conjunctive normal form (CNF) from a given first-order theory.

### A. Non-linear Real Arithmetic (NRA)

NRA is formally defined as the first-order theory of the structure  $(\mathbb{R}, 0, 1, +, \cdot, <, =)$ . This means that, in NRA, variables range over real numbers, and wlog. atomic formulas are strict inequalities  $p(x) < 0$  and equalities  $p(x) = 0$ , where the term  $p(x)$  is a polynomial with integer coefficients. We write  $p \leq 0$  and  $p \neq 0$  as shortcuts for  $\neg(-p < 0)$  and  $\neg(p = 0)$ , respectively. Throughout the paper, we only consider quantifier-free formulas, given in CNF. The satisfiability problem for NRA is decidable, and in practice most decision procedures for this theory are based on Collins' Cylindrical Algebraic Decomposition (CAD) [9].

A fundamental property of NRA is that if a formula has a solution, then it has an *algebraic solution* [10, Theorem 2.81], i.e., a solution assigning algebraic numbers to all variables. A real number is *algebraic* if it is the root of a univariate nonzero polynomial with integer coefficients; else, it is *transcendental*. For instance,  $\sqrt{2}$  is algebraic since it is a solution of the equation  $x^2 - 2 = 0$ , while  $\pi$  is transcendental. All standard decision procedures for NRA restrict the search for solutions to algebraic solutions.

We write  $\mathbb{R}_{alg}$  for the set of real algebraic numbers. A real algebraic number  $\alpha$  can be effectively represented by a triple  $(p, \ell, u)$ , where  $p$  is a univariate squarefree polynomial having  $\alpha$  as a root, and  $\ell$  and  $u$  are rational numbers such that  $\alpha$  is the only root of  $p$  in the interval  $[\ell, u]$ . Evaluating the sign of  $p$  at the midpoint  $m = \frac{\ell+u}{2}$  can be done exactly over the rationals. Whenever  $p(m) \neq 0$  (otherwise,  $\alpha = m \in \mathbb{Q}$ ), by comparing the sign of  $p(m)$  with the sign of  $p(u)$  one can determine whether  $\alpha$  lies in  $[\ell, m]$  or in  $[m, u]$ . Iterating this test refines the interval to any prescribed accuracy.

### B. Real Arithmetics with Transcendental Functions

A real-valued function  $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$  is called *transcendental* if its graph cannot be characterized by any formula in NRA; that is, there is no formula  $\varphi(x, y)$  in NRA such that, for every  $r \in \mathbb{R}^n$  and  $s \in \mathbb{R}^m$ ,  $\varphi(r, s)$  holds if and only if  $f(r) = s$ . A *Transcendental Real Arithmetic (TRA)* is understood as any first-order theory of a structure that expands NRA with (computable) transcendental functions. Classical examples of TRAs are given by the first-order theories of the structures  $(\mathbb{R}, 0, 1, +, \cdot, \sin, <, =)$  and  $(\mathbb{R}, 0, 1, +, \cdot, \exp, <, =)$ , i.e., the expansions of NRA by the sine function and the exponential function  $\exp(x) := e^x$ , respectively.

TRAs are very expressive. For example, even the univariate fragment of  $(\mathbb{R}, 0, 1, +, \cdot, \sin, <, =)$  is undecidable [3], while  $(\mathbb{R}, 0, 1, +, \cdot, \exp, <, =)$  is only known to be decidable subject to Schanuel's conjecture [4]. Even worse, solutions are often *unverifiable* [11, Section 1.3]: there is no general algorithm to decide if a given variable assignment satisfies a TRA formula. This stems from the nature of transcendental functions, whose outputs can typically only be computed to arbitrary precision, whereas the formula may be satisfied only at their exact value. For these reasons, except for procedures targeting very restricted theories (see, e.g., [12], [13]), all decision procedures for TRAs are incomplete. The procedure we describe in the next section is no exception. More precisely, our procedure is designed to solve extensions of NRA by *interval-computable total functions*, defined next.

**Definition 1** (Algebraic box). *An algebraic box in  $\mathbb{R}^n$  is the Cartesian product  $[a_1, b_1] \times \dots \times [a_n, b_n]$  of  $n$  closed intervals, where  $a_i \leq b_i$  are algebraic numbers (the extrema of the box). It is represented by the  $2n$ -tuple  $(a_1, b_1, \dots, a_n, b_n)$ . We write  $\mathcal{B}_n(\mathbb{R}_{alg})$  for the set of all algebraic boxes in  $\mathbb{R}^n$ .*

*The width of an algebraic box  $B$ , denoted  $\text{width}(B)$ , is defined as  $\max_{i=1}^n (b_i - a_i)$ . We write  $\|B\|_\infty$  for the maximum of the  $\infty$ -norm of the points in  $B$ , i.e.,  $\max_{i=1}^n \max(|a_i|, |b_i|)$ .*

**Definition 2** (Interval computable function). *A function  $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$  is said to be interval computable if there is an algorithm  $I_f: \mathcal{B}_n(\mathbb{R}_{alg}) \rightarrow \mathcal{B}_m(\mathbb{R}_{alg})$  with the following property: for every  $\delta > 0$  and  $\eta > 0$ , there is an  $\epsilon > 0$  such that, when given as input any box  $B \in \mathcal{B}_n(\mathbb{R}_{alg})$  with  $0 < \text{width}(B) < \epsilon$  and  $\|B\|_\infty < \eta$ , the algorithm  $I_f$  returns a box  $C \in \mathcal{B}_m(\mathbb{R}_{alg})$  satisfying  $\text{width}(C) < \delta$  and  $f(B) \subseteq C$ .*

Three observations are in order. First, the composition of two interval computable functions is interval computable. Second, addition, multiplication, and many transcendental functions of practical interest are interval computable, including the sine and exponential functions. For these natural functions, the algorithms  $I_f$  are effective and already available in several libraries; in particular, our tool uses the ARB library [14] for arbitrary-precision interval computation of the sine and exponential functions. Lastly, any computable constant  $c$  (e.g.,  $\pi$  or  $e$ ) can be treated as a unary interval computable function: given a box of some width  $\epsilon$ ,  $I_c$  computes an interval of width  $\epsilon$  around  $c$ . Throughout the paper, we tacitly see transcendental constants as unary functions applied to the constant 1.

**Example 1.** Let us see why the exponential function is interval computable. Consider an interval  $[-\eta, \eta]$ , for some  $\eta > 0$ , and a  $\delta > 0$ . Let  $\epsilon = \frac{\delta}{e^\eta}$ . Consider a box  $[a, b] \subseteq [-\eta, \eta]$  of positive width  $b - a < \epsilon$ . Since  $\exp$  is continuous and differentiable, by the mean value theorem  $e^b - e^a = e^\xi(b - a)$  for some  $\xi \in [a, b]$ . Since  $\exp$  is monotone, we then have  $e^b - e^a < e^\eta \epsilon = \delta$ . On input  $[a, b]$ , a simple algorithm  $I_{\exp}$  consists in computing  $p, q \in \mathbb{Q}$  such that  $e^a < p < q < e^b$ .

*$\delta$ -satisfiability:* Because of undecidability and unverifiability, symbolic computation often offers very limited support

in deciding TRAs, and must be complemented by numerical methods. A principled framework for combining these methods with formal soundness guarantees is given by the notion of  $\delta$ -satisfiability procedure from [15], which we now recall (in a slightly extended form that still allows SAT results).

Let  $\delta > 0$  be a rational (the *tolerance*). Let  $\varphi$  be a quantifier-free CNF formula from some expansion of NRA, with atomic formulas of the form  $\tau < 0$  or  $\tau = 0$ . The  $\delta$ -weakening of  $\varphi$ , denoted  $\varphi^\delta$ , is the formula obtained from  $\varphi$  by replacing the literals in each clause following the rules:  $\tau < 0 \rightarrow \tau < \delta$ ,  $\neg(\tau < 0) \rightarrow \neg(\tau < -\delta)$ ,  $\tau = 0 \rightarrow -\delta \leq \tau \leq \delta$ , and  $\neg(\tau = 0) \rightarrow \top$ . A formula  $\varphi$  is said to be  $\delta$ -satisfiable if and only if its  $\delta$ -weakening  $\varphi^\delta$  is satisfiable.

**Definition 3** ( $\delta$ -satisfiability procedure). A  $\delta$ -satisfiability procedure for an expansion of NRA is a procedure that, if it terminates on input formula  $\varphi$ , returns SAT, UNSAT, or  $\delta$ -SAT. The procedure adheres to the following soundness criterion: if it returns SAT (resp. UNSAT or  $\delta$ -SAT), then  $\varphi$  must be satisfiable (resp. unsatisfiable or  $\delta$ -satisfiable). When  $\varphi$  is both satisfiable (or unsatisfiable) and  $\delta$ -satisfiable, the procedure can return either of the two answers.

### C. Background on MCSAT

We briefly introduce MCSAT, referring the reader to [16] and [17] for more detailed presentations.

The MCSAT framework consists of a *core solver* that orchestrates a set of *theory plugins*. Each theory plugin implements a procedure for a specific logical theory. The control flow of the core solver, illustrated in Figure 1, closely follows that of CDCL. As in CDCL, the central data structure is the *trail*, which is a list of four different types of assignments:

- *Boolean decisions*. Assignments  $\ell \mapsto b$  where  $\ell$  is a literal and  $b$  is a Boolean. These are added to the trail during the *Decide* step.
- *Semantic decisions*. Assignments  $x \mapsto v$  where  $x$  is a first-order variable over some sort  $S$  (e.g.,  $\mathbb{R}$ ), and  $v$  is an element of  $S$ . These decisions are also added during the *Decide* step: MCSAT calls the theory plugin associated to  $S$  (e.g., the NRA plugin), asking a value for  $x$ .
- *Boolean propagations*. Assignments  $\ell \mapsto b$  where  $\ell$  is a literal and  $b$  is a Boolean, that are added during the *Propagate* step and derived by unit propagation.
- *Semantic propagations*. Assignments  $\ell \mapsto b$  added during the *Propagate* step by a theory plugin. Specifically, when the literal  $\ell$  contains variables already assigned in the trail, the theory plugin evaluates its truth value  $b$ .

At each *Decide* step, exactly one Boolean or semantic decision is added to the trail; this decision can only feature variables or literals that are not yet in the trail. The other steps of MCSAT are described below.

*Step Propagate*: This step saturates the trail with all available Boolean and Semantic propagations. Whenever the trail is updated, MCSAT notifies all plugins of the update. The plugins use this information to refine a *feasibility set* for each variable of their competence. For example, if  $x \mapsto 2$

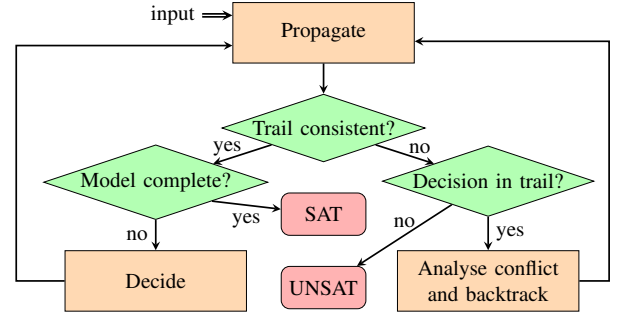


Fig. 1: The MCSAT framework.

is in the trail, and MCSAT notifies the NRA plugin that  $(x^2 - y \leq 0) \mapsto \top$  has been added, the plugin updates the feasibility set of  $y$  to exclude the interval  $(-\infty, 4]$ . The feasibility sets are used to detect conflicts, as well as to perform semantic decisions.

*Step Consistency*: A trail is *propositionally consistent* if it does not contain both  $\ell \mapsto \top$  and  $\ell \mapsto \perp$ , for any literal  $\ell$ . As semantic decisions do not involve variables in the trail, a propositionally consistent trail  $T$  can be viewed as a map (instead of a list), with  $T(t) = v$  if and only if  $t \mapsto v$  occurs in  $T$ . We write  $\text{dom}(T)$  for the domain of this map. A literal is said to be *fully-assigned* in  $T$  if  $\text{vars}(\ell) \subseteq \text{dom}(T)$ , where  $\text{vars}(\ell)$  is the set of first-order variables occurring in  $\ell$ . For a fully-assigned literal  $\ell$ , we write  $T \models \ell$  when the assignments of  $\text{vars}(\ell)$  in  $T$  satisfy the literal. The notation  $T \models \psi$  extends to Boolean combinations  $\psi$  of fully-assigned literals.

After *Propagate*, MCSAT checks the *Consistency* of the trail  $T$  by verifying that it is propositionally consistent, and by asking the plugins to check (1) that every unassigned variable has a non-empty feasibility set, and (2) that the value of fully-assigned literals in the trail is consistent with their satisfaction. Specifically, for each fully-assigned literal  $\ell$  with  $\ell \mapsto b$  in  $T$ , it must hold that  $T \models \ell$  if and only if  $b = \top$ .

*Step Completeness*: If the *Consistency* check passes, MCSAT verifies *Completeness* of the trail, i.e., each clause in the input CNF formula contains a fully-assigned literal  $\ell$  such that  $\ell \mapsto \top$  or  $(\neg\ell) \mapsto \perp$  is present in the trail (where  $\neg\ell$  is understood as  $\ell'$  if  $\ell = \neg\ell'$ ). If this is the case, MCSAT returns SAT since the trail satisfies all clauses in the input formula; otherwise it moves to the *Decide* step.

*Step Analyse conflict and backtrack*: When the *Consistency* check fails, if the trail does not contain a decision, then the formula is UNSAT. Otherwise, MCSAT must backtrack and learn a new clause  $\psi$  that is implied by the input formula  $\varphi$  and explains the conflict. If the conflict stems from Boolean reasoning (e.g., it is due to the Boolean propagations performed after a Boolean decision), MCSAT behaves as CDCL. Otherwise, it invokes the theory plugins. The plugins analyse the trail, and generate a suitable clause  $\psi$  explaining the conflict, along with the semantic or Boolean decision causing it. MCSAT conjoins  $\psi$  to  $\varphi$ , and then backtracks to before the problematic decision. When backtracking on a

semantic decision  $x \mapsto v$ , MCSAT does not necessarily decide  $x$  first upon reaching the `Decide` step.

**Example 2.** Consider the following CNF formula of NRA:

$$(x^2 = y) \wedge (y = -1 \vee y = 2) \wedge (\neg(y < 1) \vee \neg(x < 1)).$$

Let us discuss a possible run of MCSAT on this formula. The initial `Propagate` step pushes to the trail  $(x^2 = y) \mapsto \top$ , as the literal  $x^2 = y$  belongs to a unit clause. Next, the control flow reaches the `Decide` step. Suppose MCSAT performs a Boolean decision, adding  $(y = -1) \mapsto \top$  to the trail.

The algorithm now performs the `Consistency` check, which fails as the literals  $x^2 = y$  and  $y = -1$  caused the feasibility set of  $x$  to become empty. The plugin may learn the (tautological) clause  $(x^2 \neq y \vee y \geq 0)$ , and return the decision  $(y = -1) \mapsto \top$  as the cause of the conflict, together with the clause. MCSAT backtracks, updating the trail to  $(x^2 = y) \mapsto \top, (y = -1) \mapsto \perp$ . Unit propagation adds the assignment  $(y = 2) \mapsto \top$ . This causes the feasibility set of  $y$  to become the singleton  $\{2\}$ , which in turn causes the feasibility set of  $x$  to become  $\{\sqrt{2}\}$ . Applying `Decide` twice adds  $y \mapsto 2$  and  $x \mapsto \sqrt{2}$  to the trail, making it consistent and complete. MCSAT returns `SAT`.

### III. TRAs IN MCSAT

We describe our MCSAT-based procedure for the first-order theory of a structure  $(\mathbb{R}, 0, 1, +, \cdot, f_1, \dots, f_k, <, =)$  extending NRA with  $k$  interval computable transcendental functions  $f_j: \mathbb{R}^{n_j} \rightarrow \mathbb{R}$ . The only assumption we make is to have access, for each  $f_j$ , to an algorithm  $I_{f_j}$  as described in Definition 2. This general procedure is later tailored to the extension of NRA by the exponential and sine functions in Section IV.

#### A. High-level view of the procedure

As illustrated in Figure 2, our procedure starts with a pre-processing phase that constructs from the input formula  $\varphi$  an abstraction  $\varphi^\alpha$  in NRA (defined in Section III-B). The MCSAT framework is then executed on  $\varphi^\alpha$ , with (the core solver of) MCSAT querying the NRA plugin. The main component of our procedure is given by a *Transcendental Real Arithmetic plugin (TRA plugin)* that acts as an intermediary between the core solver and its NRA plugin. The TRA plugin has access to both the formula  $\varphi$  and the abstraction  $\varphi^\alpha$  (as well as information to connect the two formulas, discussed later).

In a nutshell, the TRA plugin is limited to observing the communication between MCSAT and the NRA plugin, except for when the consistency of the trail must be checked or conflicts must be analysed. For instance, if the NRA plugin determines that the trail is consistent, the TRA plugin runs its own consistency check, but over constraints from  $\varphi$  rather than  $\varphi^\alpha$ . If the check fails, the TRA plugin proposes clauses that are implied by  $\varphi$  but not by  $\varphi^\alpha$ , effectively refining  $\varphi^\alpha$  enough to trigger a conflict in the NRA plugin.

Apart from assuming that MCSAT performs the loop described in the previous section, our procedure relies on the following assumptions on the NRA plugin. The existence of

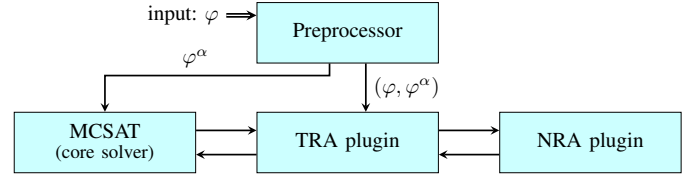


Fig. 2: Schematic view of our procedure.

a plugin satisfying them is guaranteed by the decidability of NRA, and the NRA plugin of `Yices2`, which implements the CAD-based procedure from [18], is a concrete instance.

**Assumption 1.** We assume that MCSAT with the NRA plugin is sound, complete and terminating on all instances of NRA. We moreover assume: (1) When deciding on a variable, the NRA plugin always returns an algebraic number. (2) The NRA plugin can check the consistency of any fully-assigned literal in the trail, when all variables are assigned algebraic numbers.

At its core, our procedure respects the specification of  $\delta$ -satisfiability procedure (Definition 3), but can also run in a standard SAT/UNSAT mode (as explained in Section III-G).

In practical terms, literals in the trail can take three values:  $\top$ ,  $\perp$  and  $\top^\delta$ . Except for the TRA plugin, all other theory plugins treat  $\top^\delta$  as  $\top$ . The TRA plugin can weaken literals in the trail, replacing  $\top$  with  $\top^\delta$  while performing its consistency check (explained in Section III-D). Roughly speaking, an assignment  $\ell \mapsto \top^\delta$  in the trail should be understood as “ $\ell$  is a fully-assigned literal in the trail that is true with respect to the abstraction, but whose satisfaction in the original formula is only guaranteed for its  $\delta$ -weakening”.

In the core solver of MCSAT, the only difference concerns the `Completeness` check. When this check passes (seeing  $\top^\delta$  as  $\top$ ), but a clause in the formula is only true because of a literal with value  $\top^\delta$ , MCSAT returns  $\delta$ -SAT instead of SAT.

#### B. Preprocessing: the first NRA abstraction

We discuss how the first abstraction  $\varphi^\alpha$  is built. We tag each clause in the input formula  $\varphi$  as `original`, so that the TRA plugin can distinguish literals from these clauses among those in  $\varphi^\alpha$ . These are referred to as `original` clauses and literals. Afterwards, we iteratively consider all terms  $f(t_1, \dots, t_r)$  in  $\varphi$  starting from the outermost ones, where  $f$  is a transcendental function, and perform the rewriting step

$$\varphi \longrightarrow \varphi[f_{(t_1, \dots, t_r)}^{\text{out}} / f(t_1, \dots, t_r)] \wedge \bigwedge_{i=1}^r f_{t_i}^{\text{in}} = t_i,$$

where each  $f_{t_i}^{\text{in}}$  and  $f_{(t_1, \dots, t_r)}^{\text{out}}$  are fresh real-valued variables, and  $\varphi[t' / t'']$  stands for the formula obtained from  $\varphi$  by replacing every occurrence of  $t''$  with  $t'$ . This rewriting step is applied until no transcendental function is left in the formula, and the resulting (NRA) formula is  $\varphi^\alpha$ .

The TRA plugin is initialised by providing the formulas  $\varphi$  and  $\varphi^\alpha$ , as well as maps to reconstruct terms  $f(t_1, \dots, t_r)$  from the variables  $f_{t_i}^{\text{in}}$  and  $f_{(t_1, \dots, t_r)}^{\text{out}}$ . Given a literal  $\ell$  (resp. term  $\tau$ ) occurring in  $\varphi^\alpha$ , we write  $\ell^\gamma$  (resp.  $\tau^\gamma$ ) for the literal (resp. term) obtained by replacing every  $f_{(t_1, \dots, t_r)}^{\text{out}}$

with  $f(t_1, \dots, t_r)$  and every  $f_{t_i}^{\text{in}}$  with  $t_i$ . When the literal  $\ell$  is original, we write  $\text{allvars}(\ell)$  for all the variables in  $\ell^\gamma$ , plus all variables added when constructing  $\ell$  from  $\ell^\gamma$ .

**Example 3.** Consider  $\varphi := \sin(\cos(x)) + \cos(x)^2 < 0$ . The NRA abstraction  $\varphi^\alpha$  is defined as

$$\sin_{\cos(x)}^{\text{out}} + (\cos_x^{\text{out}})^2 < 0 \wedge \sin_{\cos(x)}^{\text{in}} = \cos_x^{\text{out}} \wedge \cos_x^{\text{in}} = x.$$

The (unit) clause  $\ell := \sin_{\cos(x)}^{\text{out}} + (\cos_x^{\text{out}})^2 < 0$  is the only one tagged as `original`. The set of variables  $\text{allvars}(\ell)$  is defined as  $\{x, \sin_{\cos(x)}^{\text{in}}, \sin_{\cos(x)}^{\text{out}}, \cos_x^{\text{in}}, \cos_x^{\text{out}}\}$ .

The following lemma, whose proof is immediate, states fundamental properties of  $\varphi^\alpha$ . The TRA plugin preserves these properties as invariants throughout the procedure, while the NRA abstraction  $\varphi^\alpha$  is refined.

**Lemma 1.** *The formula  $\varphi^\alpha$  satisfies the following properties. (1) Every variable in  $\varphi$  occurs in  $\varphi^\alpha$ . (2) Every solution to  $\varphi$  can be extended into a solution to  $\varphi^\alpha$ .*

After constructing  $\varphi^\alpha$  and initializing the TRA plugin, MCSAT runs on  $\varphi^\alpha$  as described in Section II-C.

**Remark 1.** MCSAT starts with a `Propagate` step, during which unit propagation adds to the trail all equalities  $f_t^{\text{in}} = t$  that arise when rewriting  $\varphi$  into  $\varphi^\alpha$ . These equalities remain in the trail throughout the execution of MCSAT.

We now detail the role of the TRA plugin at each step of the MCSAT framework.

### C. Steps: `Decide` and `Propagate`

In these two steps, the role of the TRA plugin is limited to forwarding data from MCSAT to the NRA plugin, and returning the results computed by the latter back to MCSAT. Note that, by Assumption 1, this implies that all semantic decisions assign algebraic values to variables.

### D. Step: `Consistency`

Recall that during this step, the theory plugins verify (1) that the feasibility sets they maintain for the unassigned variables are non-empty, and (2) that the values of fully-assigned literals in the trail are consistent with their satisfaction.

The TRA plugin begins this step by forwarding the request to the NRA plugin, which performs its own consistency check. Should the trail be found inconsistent at this stage, the NRA plugin raises a conflict that the TRA plugin propagates back to MCSAT. Otherwise, the TRA plugin proceeds with an additional consistency check.

The additional check does not concern feasibility sets: the TRA plugin relies on the feasibility sets of the NRA plugin, which, by definition of  $\varphi^\alpha$ , are over-approximations of the possible values that variables can take. While it is sometimes possible to refine these approximations beyond those of the NRA plugin (see, e.g., [12]), the undecidability status of even the univariate extension of NRA by the sine function [3] (which is interval computable) prevents the determination of exact feasibility sets in general.

---

### Algorithm 1 TRA $\delta$ -consistency check.

---

**Fixed:** A tolerance  $\delta > 0$ .

**Input:** ( $\ell$ : literal,  $T$ : trail) such that  $\ell$  is from an original clause of  $\varphi^\alpha$ , and  $\text{allvars}(\ell) \subseteq \text{dom}(T)$ .

**Output:**  $b \in \{\top, \perp, \top^\delta\}$  respecting Definition 4.

- 1: **if**  $\ell^\gamma$  contains no transcendental function **then return**  $T(\ell)$
- 2: **let**  $\tau \sim 0$  be the atomic formula in  $\ell$ , where  $\sim \in \{<, =\}$
- 3: **let**  $[i, j] \in \mathcal{B}_1(\mathbb{R}_{\text{alg}})$  s.t.  $0 < j - i < \delta$  and  $T \models i \leq \tau^\gamma \leq j$   
 $\triangleright [i, j]$  computed bottom-up by building boxes for all subterms of  $\tau^\gamma$ , using the algorithms from Definition 2
- 4: **return**  $b \in \{\top, \perp, \top^\delta\}$  computed using the table:

| $\ell \backslash [i, j]$ | $j < 0$ | $i > 0$ | $i = j = 0$ | $0 = i < j$   | $i < 0 \leq j$ |
|--------------------------|---------|---------|-------------|---------------|----------------|
| $\tau < 0$               | $\top$  | $\perp$ | $\perp$     | $\perp$       | $\top^\delta$  |
| $\tau = 0$               | $\perp$ | $\perp$ | $\top$      | $\top^\delta$ | $\top^\delta$  |
| $\neg(\tau < 0)$         | $\perp$ | $\top$  | $\top$      | $\top$        | $\top^\delta$  |
| $\neg(\tau = 0)$         | $\top$  | $\top$  | $\perp$     | $\top^\delta$ | $\top^\delta$  |

---

The consistency check of the TRA plugin is therefore focused solely on the second of the points above: verifying that the values of fully-assigned literals in the trail are consistent with their satisfaction. Because this verification problem is undecidable (as explained in Section II-B), we perform it only “up to  $\delta$ ”. We call this the *TRA  $\delta$ -consistency check*:

**Definition 4.** A TRA  $\delta$ -consistency check is a procedure that given an original literal  $\ell$  and a trail  $T$  such that  $\text{allvars}(\ell) \subseteq \text{dom}(T)$  returns  $\top$ ,  $\perp$  or  $\top^\delta$  following the criterion: if it returns  $\top$ , then  $T \models \ell^\gamma$ , if it returns  $\perp$ , then  $T \not\models \ell^\gamma$ , and if it returns  $\top^\delta$ , then  $T \models (\ell^\gamma)^\delta$ .

Algorithm 1, which we discuss in a moment, implements the TRA  $\delta$ -consistency check. Following Definition 4, we see that there are two cases in which the TRA plugin will not perform this check on a literal  $\ell$  occurring in the trail:

- When  $\ell$  is not original. The decision is pragmatic: clauses will only be added to improve the NRA abstraction and therefore soundness is still guaranteed.
- If some variable in the set  $\text{allvars}(\ell)$  is unassigned in the trail. In this case, the TRA plugin places  $\ell$  in a *watchlist* and defers its consistency check until all variables in  $\text{allvars}(\ell)$  are assigned (note: this must occur for the *Completeness* check to pass).

Consider thus an assignment  $\ell \mapsto b'$  in the trail  $T$ , such that  $\ell$  is original, all variables in  $\text{allvars}(\ell)$  are assigned in  $T$ , and that passes the consistency check of the NRA plugin, i.e., we have  $T \models \ell$  if and only if  $b' = \top$ . Algorithm 1 works as follows. First, if  $\ell^\gamma$  contains no transcendental function, it returns  $b'$ . Indeed, in this case  $\ell = \ell^\gamma$ , and the NRA plugin has already verified  $\ell \mapsto b'$ . Otherwise, let  $\tau$  be the term occurring in  $\ell$  (as in line 2). Since interval computable functions are closed under composition,  $\tau^\gamma$  is interval computable. The

algorithm computes an algebraic box  $[i, j] \in \mathcal{B}_1(\mathbb{R}_{alg})$  of width less than  $\delta$ , such that  $T \models i \leq \tau^\gamma \leq j$ . This interval is computed bottom-up, computing boxes  $I(\rho)$  for every subterm  $\rho$  of  $\tau^\gamma$ , starting from the innermost subterms. Interval computability ensures that by choosing sufficiently small intervals around the values of variables in  $\tau^\gamma$  and the constants 0 and 1, the final interval  $I(\tau^\gamma) = [i, j]$  will have width less than  $\delta$ .<sup>2</sup> Algorithm 1 terminates by performing interval analysis, following the table in line 4. Note that  $\top^\delta$  is only returned in cases where  $i$  and  $j$  are neither both positive, both negative, nor both zero.

**Lemma 2.** *Algorithm 1 respects its specification.*

Returning to the assignment  $\ell \mapsto b'$ , let  $b$  denote the output of Algorithm 1 on input  $\ell$  and  $T$ . After performing the TRA  $\delta$ -consistency check, the TRA plugin completes its consistency step on  $\ell$  as follows:

- 1: **if**  $b \in \{\top, \perp\}$  and  $b \neq b'$  **then raise** conflict
- 2: **else if**  $b = \top^\delta$  and  $b' = \top$  **then**
- 3:     update  $T$ : replace  $\ell \mapsto \top$  with  $\ell \mapsto \top^\delta$

**Remark 2.** If the above snippet does not raise a conflict, the assignment  $\ell \mapsto b''$  (where  $b''$  is  $b'$  or  $\top^\delta$ ) satisfies the following properties:

- If  $b'' = \top$ , then  $T \models \ell$  and  $T \models \ell^\gamma$ .
- If  $b'' = \top^\delta$ , then  $T \models \ell$  and  $T \models (\ell^\gamma)^\delta$ .
- If  $b'' = \perp$ , then  $T \not\models \ell$ .

Note that this matches the description of  $\top^\delta$  given in Section III-A: the literal  $\ell$  from  $\varphi^\alpha$  is true, while the literal  $\ell^\gamma$  from the original TRA formula  $\varphi$  need not be true, only its  $\delta$ -weakening is guaranteed to be.

One might wonder about the asymmetry between  $\top$  and  $\perp$ , and whether a more refined treatment of  $\perp$  could be given by introducing a value  $\perp^\delta$  with semantics “ $T \not\models \ell$  and  $T \models (\ell^\gamma)^\delta$ ”. The short answer is that, since MCSAT is based on propositional logic, introducing  $\perp^\delta$  alone does not yield a better procedure: the solver would still need to treat it as  $\perp$ , except during the Completeness check. At that stage, however, no clause can feature only literals assigned to  $\perp$  and  $\perp^\delta$ , since such a situation would have triggered a conflict at the Boolean level during the Consistency check.

#### E. Step: Analyse conflict and backtrack

Suppose the TRA plugin raises a conflict, causing MCSAT to enter the Analyse conflict and backtrack step (when the trail contains a decision) and query the TRA plugin in return. If the conflict originates from the NRA plugin, the TRA plugin lets the NRA plugin resolve it, again acting as an intermediary. Otherwise, the conflict arose following the TRA  $\delta$ -consistency check. In particular, there is an assignment  $\ell \mapsto b'$  in the trail  $T$  with the following properties:

<sup>2</sup>A naïve way of computing these “sufficiently small intervals” is to start with unit-length intervals around each variable, compute  $[i, j]$ , and, if  $j - i \geq \delta$ , halve the intervals and repeat. This is possible because, as described at the end of Section II-A, the interval around a real algebraic number assigned to a variable can be refined to arbitrary accuracy.

---

#### Algorithm 2 TRA refinement.

---

**Input:**  $(\ell: \text{literal}, T: \text{trail})$  such that  $\ell$  is from an original clause of  $\varphi^\alpha$ ,  $\text{allvars}(\ell) \subseteq \text{dom}(T)$ , and Algorithm 1 on input  $(\ell, T)$  returns  $b \in \{\top, \perp\}$  with  $b \neq T(\ell)$ .

**Output:** A formula  $\psi$  in NRA such that  $T \not\models \psi$  and every solution to  $\varphi$  can be extended to a solution of  $\varphi^\alpha \wedge \psi$ .

- 1: **let**  $\tau \sim 0$  with  $\sim \in \{<, =\}$  be the atomic formula in  $\ell$
  - 2: For each subterm  $\rho$  of  $\tau^\gamma$  ( $\tau^\gamma$  included) recompute the box  $[i_\rho, j_\rho] \in \mathcal{B}_1(\mathbb{R}_{alg})$  used in Algorithm 1 to compute  $[i, j]$
  - 3:  $\psi \leftarrow \top$
  - 4: **for**  $\eta := f(t)$  subterm of  $\tau^\gamma$  with  $f$  transcendental **do**
  - 5:     **let**  $\mathcal{T}$  be the set of terms in the vector  $t$
  - 6:      $\psi \leftarrow \psi \wedge ((\bigwedge_{\rho \in \mathcal{T}} i_\rho \leq f_\rho^{\text{in}} \leq j_\rho) \Rightarrow i_\eta \leq f_t^{\text{out}} \leq j_\eta)$
  - 7: **return**  $\psi$
- 

- $\ell$  is original, and  $\text{allvars}(\ell) \subseteq \text{dom}(T)$ .
- $T \models \ell$ , but  $T \not\models \ell^\gamma$ . More precisely, Algorithm 1 returned  $b \in \{\perp, \top\}$  on input  $\ell$  and  $T$ , with  $b \neq b'$ .

The TRA plugin uses  $\ell$  in order to refine  $\varphi^\alpha$  so that it continues to satisfy the properties in Lemma 1 while also triggering a conflict in the NRA plugin.

The refinement consists in conjoining to  $\varphi^\alpha$  the formula  $\psi$  computed by Algorithm 2. In a nutshell, this algorithm considers all subterms  $f(t_1, \dots, t_r)$  of  $\ell^\gamma$  where  $f$  is a transcendental function, and adds (Line 6) entailments stating that if the value assigned to  $f_{t_j}^{\text{in}}$  lies in  $I(t_j)$  for every  $j \in [1..r]$ , then the value assigned to  $f_{(t_1, \dots, t_r)}^{\text{out}}$  must lie in  $I(f(t_1, \dots, t_r))$ , where  $I(\rho)$  denotes the interval computed for subterm  $\rho$  during the execution of Algorithm 1 on  $\ell$  and  $T$ .

**Lemma 3.** *Algorithm 2 respects its specification.*

*Proof idea.* To prove that  $T \not\models \psi$  one reasons by contradiction. Let  $\tau \sim 0$  be the atomic formula in  $\ell$ . We assume that  $T \models \psi$ , and under this assumption show that  $T \models \rho \in I(\rho)$  holds for each subterm  $\rho$  of the term  $\tau$  ( $\tau$  included). The induction relies on Remark 1 to handle function composition. This implies  $T \models \tau \in I(\tau)$ , which in turn implies  $b = T(\ell)$ , contradicting the hypotheses on the input of Algorithm 2.

For the proof that every solution to  $\varphi$  can be extended into a solution to  $\varphi^\alpha \wedge \psi$ , we rely on the fact that  $\psi$  over-approximates the transcendental functions occurring in  $\tau$ .  $\square$

After computing  $\psi$ , the TRA plugin identifies a conjunct  $(\bigwedge_{\rho \in \mathcal{T}} i_\rho \leq f_\rho^{\text{in}} \leq j_\rho) \Rightarrow i_\eta \leq f_t^{\text{out}} \leq j_\eta$  of this formula that is not satisfied by  $T$ . This reduces to evaluating inequalities between algebraic numbers, which can be done again by interval analysis. Note that the problem is in fact solvable in polynomial time [19].

Let  $x$  be the variable in the identified conjunct that was decided last in the trail  $T$ . The TRA plugin returns to MCSAT the formula  $\psi$ , to be conjoined to  $\varphi^\alpha$  (refining the abstraction),

along with the variable  $x$  to backtrack. Before doing so, however, the TRA plugin performs some bookkeeping to ensure that the watchlist of literals waiting for the  $\delta$ -consistency check, and their values in the trail, will remain consistent with the trail after backtracking. More precisely, the plugin will re-add to the watchlist every literal  $\ell$  on the trail that, after backtracking  $x$ , will have at least one variable in  $\text{allvars}(\ell)$  become unassigned. Moreover, for any such literal, any trail assignment  $\ell \mapsto \top^\delta$  is set back to  $\ell \mapsto \top$ .

**Remark 3.** Let us assume that the variable  $x$  is  $f_\rho^{\text{in}}$  for some  $\rho$  (the case of  $f_t^{\text{out}}$  is similar). After backtracking  $f_\rho^{\text{in}}$ , unit propagation will add either  $\neg(i_\rho \leq f_\rho^{\text{in}})$  or  $\neg(f_\rho^{\text{in}} \leq j_\rho)$  to the trail, ensuring that the antecedent of the identified conjunct is falsified. Then, should the NRA plugin be asked to decide a value for  $f_\rho^{\text{in}}$ , it will return a value different from the one assigned before backtracking.

#### F. Soundness of the procedure

We show that ours is a  $\delta$ -satisfiability procedure:

**Theorem 1** (Soundness). *Let  $\varphi$  be a TRA formula and  $\delta > 0$  be a tolerance. If MCSAT with the TRA plugin in  $\delta$ -mode returns SAT,  $\varphi$  is satisfiable; if it returns UNSAT,  $\varphi$  is unsatisfiable; if it returns  $\delta$ -SAT,  $\varphi$  is  $\delta$ -satisfiable.*

*Proof sketch.* **SAT case:** If MCSAT returns SAT, the trail  $T$  assigns a value to all variables in  $\varphi^\alpha$ , and every clause has a literal  $\ell$  assigned to  $\top$ . When  $\ell$  is original, Remark 2 gives  $T \models \ell^\gamma$ . Hence, every original clause in  $\varphi$  has a literal satisfied by  $T$ , and  $\varphi$  is therefore satisfiable.

**$\delta$ -SAT case:** This case is similar to the previous one, but now there is a clause in  $\varphi^\alpha$  with no literal assigned to  $\top$ , but with a literal  $\ell$  assigned to  $\top^\delta$ . Remark 2 gives  $T \models (\ell^\gamma)^\delta$ , and  $\varphi$  is found to be  $\delta$ -satisfiable.

**UNSAT case:** In this case,  $\varphi^\alpha$  is found to be unsatisfiable. By the second property in Lemma 1, which is preserved by Lemma 3 (Algorithm 2),  $\varphi$  is unsatisfiable as well.  $\square$

#### G. More on $\delta$ -satisfiability

The notion of  $\delta$ -satisfiability procedure was introduced in [15] to provide theoretical grounding for the use of numerical methods in decision procedures. Different procedures may exploit the flexibility this notion offers in different ways. In our setting, we only employ  $\delta$ -weakening when checking the consistency of literals involving transcendental functions (Algorithm 1). This means, for example, that we never relax literals that are in NRA. We view our use of  $\delta$ -weakening as more of an *algorithmic choice* than a *theoretical necessity*. For instance, suppose  $\varphi^\alpha$  has already been refined enough to become inconsistent, although MCSAT and the NRA plugin have not yet detected this. At this stage, any computation performed by the TRA plugin is superfluous. Employing  $\delta$ -weakening yields a terminating consistency check, at the expense of forgoing the exact consistency of certain constraints (which, owing to undecidability, may in any case be unattainable). This guarantees that MCSAT and the NRA plugin will eventually be able to establish UNSAT.

Note that even the definition of TRA  $\delta$ -consistency check (Def. 4) allows some flexibility in the output. For example, when the interval  $[i, j]$  computed for a literal  $\tau < 0$  is included in  $(0, \delta)$ , a  $\delta$ -consistency check may return either  $\perp$  or  $\top^\delta$ . However, the table that we use in Algorithm 1, which returns  $\perp$  in this case, is the most precise possible. This guarantees that, within a single consistency check, an inconsistency output (that leans toward proving UNSAT) is always preferred over a  $\delta$ -consistency one (which leans toward proving  $\delta$ -SAT). We leave as future work determining whether other choices for this table may yield better performance in practice. Less precise tables could in principle allow some UNSAT instances to be identified sooner (when coarser lemmas suffice) but may cause some UNSAT instances to be declared  $\delta$ -SAT instead.

Our  $\delta$ -satisfiability procedure naturally extends to a standard SAT/UNSAT procedure. It suffices to iteratively run the  $\delta$ -satisfiability procedure starting with a large value of  $\delta$  (e.g.,  $\delta = 1$ ), halving  $\delta$  and restarting when finding  $\delta$ -SAT. Thus, in its simplest form, the loop looks as follows:

```

while true do
  res ← check( $\varphi, \delta$ )
  if res =  $\delta$ -SAT then  $\delta \leftarrow \frac{\delta}{2}$  else return res

```

Iterations of this loop can be done while preserving the current abstraction  $\varphi^\alpha$ , since the learned lemmas are independent of  $\delta$ . Decision heuristics (based, e.g., on caching [20]) can further improve efficiency. Since  $\delta$ -SAT is never returned, Theorem 1 guarantees that this SAT/UNSAT mode is sound with respect to the standard notion of satisfiability.

The mode of operation ultimately depends on the application. In fields such as control theory, a margin of safety arises naturally (from, e.g., modelling measurement errors in sensors), making it reasonable to conflate  $\delta$ -SAT with SAT or with UNSAT (depending on the system being modelled). In theorem proving, a  $\delta$ -SAT answer carries effectively no information; only exact SAT or UNSAT answers are meaningful.

## IV. SPECIALIZED TREATMENT OF THE SINE AND EXPONENTIAL FUNCTIONS

The previous section outlined a general procedure for TRAs based solely on interval computability. Here, we show how additional assumptions can be used to refine this base procedure. We focus on the sine and exponential functions, though the approach extends in a modular way to other analytic functions. Formally, we now consider the structure  $(\mathbb{R}, 0, 1, \pi, e, +, \cdot, \sin, \exp, f_1, \dots, f_k, <, =)$ , where  $f_1, \dots, f_k$  are interval computable functions. (We recall that we see the constants  $\pi$  and  $e$  as interval-computable unary constant functions applied to the constant 1.)

#### A. Refining clause learning

The main improvement<sup>3</sup> to the procedure concerns refinements (i.e., the conjunctions added in line 6 of Algorithm 2).

<sup>3</sup>A further improvement, omitted for lack of space, consists of adding clauses (none marked as *original*) to the initial abstraction  $\varphi^\alpha$ , limiting the values of “in” and “out” variables. For example, we add constraints such as  $-1 \leq \sin_t^{\text{out}} \leq 1$ ,  $1 + \exp_t^{\text{in}} \leq \exp_t^{\text{out}}$ , and  $3.14 < \pi_1^{\text{out}} < 3.15$ .

Let  $h$  be either  $\sin$  or  $\exp$ , and consider a conjunct of the form

$$\gamma(h_t^{\text{in}}, h_t^{\text{out}}) := (i_{\text{in}} \leq h_t^{\text{in}} \leq j_{\text{in}} \Rightarrow i_{\text{out}} \leq h_t^{\text{out}} \leq j_{\text{out}}), \quad (1)$$

conjoined to  $\psi$  at this line. Our goal is to strengthen  $\psi$  by replacing  $\gamma$  with a formula  $\chi$  that entails it, while preserving the specification of Algorithm 2. To obtain a substantial strengthening of  $\gamma$ , we furthermore require  $\chi$  to satisfy:

**P:** for every  $[a, b] \subseteq [\lfloor i_{\text{in}} \rfloor - 1, \lfloor j_{\text{in}} \rfloor + 1]$  with  $b - a \leq j_{\text{in}} - i_{\text{in}}$ , where  $\lfloor k \rfloor$  stands for the integer closest to  $k$ , the possible values  $h_t^{\text{out}}$  can take in a solution to  $a \leq h_t^{\text{in}} \leq b \wedge \chi$  all lie in an interval of width at most  $j_{\text{out}} - i_{\text{out}}$ .

Property P guarantees that  $\chi$  approximates  $h$  well throughout an interval of width (at least) 2. We now define the formula  $\chi$  for the sine and exponential functions.

### B. Definition of $\chi$ for the sine function

For the sine function, we define  $\chi$  through Taylor approximations. Let  $f: \mathbb{R} \rightarrow \mathbb{R}$  be an infinitely differentiable function, and  $c \in \mathbb{R}$ . We recall that the Taylor expansion of  $f$  centred at  $c$  is the infinite sum  $\sum_{k=0}^{\infty} \frac{f^{(k)}(c)}{k!} (x-c)^k$ , where  $f^{(k)}$  denotes the  $k$ th derivative of  $f$ . The truncation to the first  $n$  terms of the sum is the  $n$ th Taylor polynomial of  $f$  centred at  $c$ .

For  $r \in \mathbb{Q}$  and  $d \in \mathbb{N}$ , define  $(T_r^d \sin)(x)$  as the term  $\sum_{k=0}^d \frac{\sin^{(k)}(r \cdot \pi)}{k!} (x - r \cdot \pi_1^{\text{out}})^k$ . It has the following properties:

- 1) If  $r$  is of the form  $2 \cdot j + \frac{1}{2}$  or  $2 \cdot j + \frac{3}{2}$ , for some  $j \in \mathbb{Z}$ , then  $\sin^{(k)}(r \cdot \pi)$  is in  $\{-1, 0, 1\}$ .
- 2) Assuming that  $\pi_1^{\text{out}}$  is assigned the value  $\pi$ , then

$$\begin{aligned} (T_{2 \cdot j + \frac{1}{2}}^{4 \cdot n + 2} \sin)(x) &\leq \sin(x) \leq (T_{2 \cdot j + \frac{1}{2}}^{4 \cdot n} \sin)(x), \text{ and} \\ (T_{2 \cdot j + \frac{3}{2}}^{4 \cdot n} \sin)(x) &\leq \sin(x) \leq (T_{2 \cdot j + \frac{3}{2}}^{4 \cdot n + 2} \sin)(x), \end{aligned}$$

for all  $j \in \mathbb{Z}$ ,  $n \in \mathbb{N}$ , and  $x \in \mathbb{R}$ .

- 3) For every  $r \in \mathbb{Q}$ ,  $y = (T_r^d \sin)(x)$  tends to the graph of  $y = \sin(x)$  as  $d \rightarrow \infty$  and  $\pi_1^{\text{out}} \rightarrow \pi$ .

Consider  $\gamma(\sin_t^{\text{in}}, \sin_t^{\text{out}})$  from Equation (1). Let  $p_\ell, p_u \in \mathbb{Z}$  be such that  $[\lfloor i_{\text{in}} \rfloor - 1, \lfloor j_{\text{in}} \rfloor + 1] \subseteq [2 \cdot \pi \cdot p_\ell, 2 \cdot \pi \cdot p_u]$  and  $p_u - p_\ell$  is minimal. For  $\ell < u \in \mathbb{Q}$  with  $\pi \in [\ell, u]$  and  $n \in \mathbb{N}$ , let  $\chi_{[\ell, u]}^n(\sin_t^{\text{in}}, \sin_t^{\text{out}}, \pi_1^{\text{out}})$  be the formula given by:

$$\begin{aligned} &\ell < \pi_1^{\text{out}} < u \\ &\wedge \bigwedge_{j=p_\ell}^{p_u} (T_{2 \cdot j + \frac{1}{2}}^{4 \cdot n + 2} \sin)(\sin_t^{\text{in}}) \leq \sin_t^{\text{out}} \leq (T_{2 \cdot j + \frac{1}{2}}^{4 \cdot n} \sin)(\sin_t^{\text{in}}) \\ &\wedge \bigwedge_{j=p_\ell}^{p_u} (T_{2 \cdot j + \frac{3}{2}}^{4 \cdot n} \sin)(\sin_t^{\text{in}}) \leq \sin_t^{\text{out}} \leq (T_{2 \cdot j + \frac{3}{2}}^{4 \cdot n + 2} \sin)(\sin_t^{\text{in}}). \end{aligned}$$

By Items 1 and 2,  $\chi_{[\ell, u]}^n$  is a satisfiable formula of NRA. Moreover, Item 3 implies that, as  $n$  increases and the interval  $[\ell, u]$  is taken as a tighter approximation of  $\pi$ , the set of solutions to  $\chi_{[\ell, u]}^n$  converges to that of the TRA formula  $\pi_1^{\text{out}} = \pi \wedge \sin_t^{\text{out}} = \sin(\sin_t^{\text{in}})$ . We may thus define  $\chi := \chi_{[\ell, u]}^n$  for  $n$  sufficiently large and  $[\ell, u]$  sufficiently tight rational approximation of  $\pi$ . This construction is effective:

**Lemma 4.** *One can algorithmically compute  $\ell, u$  and  $n$  such that the formula  $\chi_{[\ell, u]}^n$  entails  $\gamma$  and satisfies Property P. Furthermore, substituting  $\gamma$  with  $\chi_{[\ell, u]}^n$  in the output formula  $\psi$  of Algorithm 2 preserves all properties of  $\psi$  guaranteed by the specification of that algorithm.*

### C. Definition of $\chi$ for the exponential function

To define  $\chi$  for the exponential function, we can still rely on Taylor approximations for lower bounds. For  $c \in \mathbb{Z}$  and  $d \in \mathbb{N}$ , define  $(T_c^d \exp)(x)$  as the term  $(e_1^{\text{out}})^c \cdot \sum_{k=0}^d \frac{1}{k!} (x-c)^k$ . When  $e_1^{\text{out}}$  is assigned the value  $e$ , this term corresponds to the  $d$ -th Taylor polynomial of  $\exp$  centred at  $c$ , and we have  $(T_c^{2n+1} \exp)(x) \leq \exp(x)$  for every  $n \in \mathbb{N}$ ,  $c \in \mathbb{Z}$  and  $x \in \mathbb{R}$ .

Taylor approximations are inadequate for upper-bounding the exponential function, as no finite polynomial approximation can serve as a global upper bound for a super-polynomial function. For this purpose, we use Padé approximants instead, which are generalizations of Taylor polynomials by rational functions [21].

Given  $c \in \mathbb{Z}$  and  $d \in \mathbb{N}$ , define the term

$$(P_c^d \exp)(x) := (e_1^{\text{out}})^c \cdot \frac{Q_d(c, x)}{Q_d(x, c)},$$

where  $Q_d(x, y) := \sum_{k=0}^d \frac{(2d-k)!}{(d-k)!d!k!} (y-x)^k$ . When  $e_1^{\text{out}}$  is assigned the value  $e$ , this term corresponds to the  $d$ -th Padé approximant of  $\exp$  centred at  $c$ , and has the following well-known properties for every  $n \in \mathbb{N}$  and  $c \in \mathbb{Z}$ :

- 1) The polynomial  $Q_{2n+1}(x, c)$  has a single root  $r > c$ , and  $\exp(x) \leq (P_c^{2n+1} \exp)(x)$  for every  $x \in [c, r)$ .
- 2)  $\exp(x) \leq (P_c^{2n} \exp)(x)$  for every  $x \in (-\infty, c]$ . (In this case,  $Q_{2n}(x, c)$  has no roots.)
- 3)  $y = (P_c^d \exp)(x)$  tends to the graph of  $y = \exp(x)$  as  $d \rightarrow \infty$  (and the root  $r$  tends to infinity).

Let  $\gamma(\exp_t^{\text{in}}, \exp_t^{\text{out}})$  be as in Equation (1), and  $c := \lfloor j_{\text{in}} \rfloor + 1$ . For  $\ell < u \in \mathbb{Q}$  with  $e \in [\ell, u]$ , and  $n \in \mathbb{N}$ , let  $\chi_{[\ell, u]}^n$  be the formula in variables  $\exp_t^{\text{in}}, \exp_t^{\text{out}}$  and  $e_1^{\text{out}}$  given by:

$$\begin{aligned} &\ell < e_1^{\text{out}} < u \wedge (T_c^{2n+1} \exp)(\exp_t^{\text{in}}) \leq \exp_t^{\text{out}} \wedge \\ &(\exp_t^{\text{in}} \leq c \Rightarrow \exp_t^{\text{out}} \leq (P_c^{2n} \exp)(\exp_t^{\text{in}})) \wedge \\ &\exists r (Q_{2n+1}(r, c) = 0 \wedge (c \leq \exp_t^{\text{in}} < r \Rightarrow \\ &\quad \exp_t^{\text{out}} \leq (P_c^{2n+1} \exp)(\exp_t^{\text{in}}))). \end{aligned}$$

One can show that Lemma 4 also holds for the formula  $\chi_{[\ell, u]}^n$  above, with respect to  $\gamma(\exp_t^{\text{in}}, \exp_t^{\text{out}})$ . It thus suffices to compute  $\ell, u$  and  $n$  as given in Lemma 4, and define  $\chi := \chi_{[\ell, u]}^n$ .

## V. IMPLEMENTATION AND EXPERIMENTAL EVALUATION

We implemented our procedure, restricted to the structure  $(\mathbb{R}, 0, 1, \pi, e, +, \cdot, \sin, \exp, <, =)$ , as an open-source plugin for Yices2, relying on its MCSAT infrastructure and NRA plugin. We use the ARB library [14] for the arbitrary-precision interval arithmetic required by the  $\delta$ -consistency check. Below, we refer to the resulting tool as Yices-TRA.

For the benchmark, we use the same instances and time limit as [6]: 2512 instances with a time limit of 1000 seconds, and no memory limit. We refer to [6] for further details on the instances. The experiments were run using Yices-TRA at commit f0c24eba. A Zenodo record of the experimental evaluation is available at DOI 10.5281/zenodo.21705986.

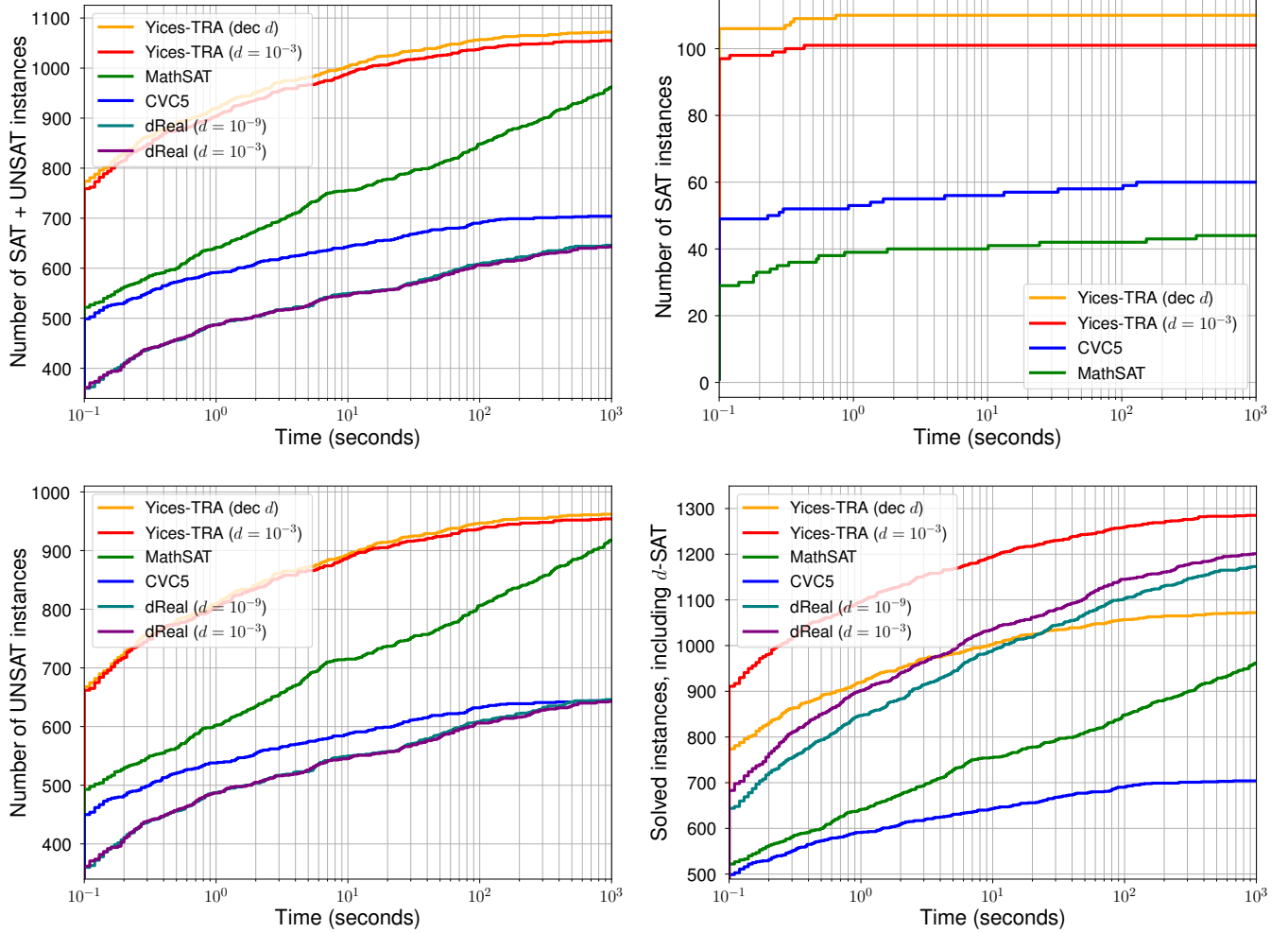


Fig. 3: Cactus plots of the results of the benchmark.

We performed two runs of Yices-TRA, one with  $\delta$  fixed at  $10^{-3}$  and one in the SAT/UNSAT mode with decreasing  $\delta$ . The latter is denoted as Yices-TRA (dec  $\delta$ ) in plots. We compared our tool with the SMT solvers MathSAT (v5.6.15) and cvc5 (v1.3.2), and with the  $\delta$ -satisfiability tool dReal (v4.21.06.2), with tolerances  $\delta = 10^{-3}$  and  $\delta = 10^{-9}$ . The two runs of dReal performed very similarly, suggesting that varying  $\delta$  would not affect the results significantly. Section VI gives more information on the methods these tools implement.

The results are presented using survival plots in Figure 3, and show the competitiveness of our approach. Runs taking less than 0.1 seconds are normalized to 0.1 seconds for clarity. With  $\delta$  fixed, our tool reported 101 SAT, 954 UNSAT and 230  $\delta$ -SAT, outperforming all other tools in each category. The SAT/UNSAT mode classified a further 9 SAT and 8 UNSAT instances that were reported as  $\delta$ -SAT by the fixed  $\delta$  run. In total, our tool classified as SAT or UNSAT 218 instances that remained unsolved by the other three tools.

Among the other tools, cvc5 performs better on SAT instances and MathSAT on UNSAT ones. Interestingly, although Yices-TRA solves significantly more UNSAT instances than

MathSAT within short time limits (e.g., less than 10s), the latter tool almost catches up at 1000s, falling short of just 44 benchmarks. This seems to suggest that the linear lemmas MathSAT uses (see *incremental linearization* in Section VI) converge slower than our NRA lemmas, but might scale better.

The first plot counts only instances classified as SAT or UNSAT, excluding  $\delta$ -SAT. (For dReal, this means counting only UNSAT instances; the tool cannot return SAT.) The last plot also counts  $\delta$ -SAT instances as solved. We observe that the ratio of  $\delta$ -SAT to solved instances differs markedly between Yices-TRA and dReal. Specifically, with  $\delta = 10^{-3}$ , dReal classifies 643 instances as UNSAT and 558 as  $\delta$ -SAT; that is, 46% of the instances solved by dReal are classified as  $\delta$ -SAT. By contrast, Yices-TRA classifies only 18% of the instances it solves as  $\delta$ -SAT. Looking more closely at the  $\delta$ -SAT instances, Yices-TRA returned UNSAT for 226 instances that dReal classified as  $\delta$ -SAT, while only 9 instances classified as  $\delta$ -SAT by Yices-TRA were classified as UNSAT by dReal. This suggests that, while  $\delta$ -SAT might intuitively give the impression that a formula is “probably satisfiable”, in practice it often is not.

## VI. RELATED WORK

*Incremental Linearization.* The idea of incremental linearization [6] is to lazily approximate all non-linear functions with linear constraints, and solve the approximation with linear solvers. If the approximation is found unsatisfiable, so is the original formula. Otherwise, the model is either actual or spurious. Linear lemmas are added to exclude the spurious model. `MathSAT` and `cvc5` implement this procedure. Our procedure is also an abstraction-refinement approach, but uses NRA lemmas with higher-degree polynomials. A benefit of this approach is that we can return irrational solutions, while linear solvers are limited to rational ones. Another distinction is that both `MathSAT` and `cvc5` are based on CDCL(T). We instead work in the MCSAT framework, which is known to perform particularly well with NRA, as it allows CAD computations to be performed locally near conflicting assignments.

*Interval Constraint Propagation (ICP).* ICP is a branch-and-prune technique [22], [23]. Given a starting box, it uses interval arithmetic to determine constraint signs over the box. If all constraint signs are constant, ICP returns whether the formula is satisfied. Else, it splits the box into smaller boxes and recurs. The results of the recursive calls are computed lazily: as long as one answers true, so does ICP. It is implemented in `iSAT3` [24] and (under the  $\delta$ -satisfiability framework) in `dReal`. In our tool, we also use interval arithmetic, but its use is limited to computing the over-approximation of a function over a single point.

*$\delta$ -satisfiability.* We have already introduced  $\delta$ -satisfiability in Section II-B. The tool `dReal` is tailored for this relaxed notion of satisfiability, and can only return UNSAT or  $\delta$ -SAT. This differs from the  $\delta$ -consistency mode of our tool, which can still answer SAT, and treats NRA constraints exactly, since they are not affected by the TRA consistency check. The `kSMT` calculus [25] combines  $\delta$ -sat with linearization (however, the tool does not support transcendental functions).

*Deductive methods.* The `MetiTarski` [26] theorem prover can prove unsatisfiability for some formulas in TRAs. It replaces transcendental functions with user-defined lower and upper bounds, and relies on decision procedures for NRA to prove unsatisfiability. Compared to our method, `MetiTarski` is not able to prove satisfiability, and it only relies on the initial abstraction provided by the user.

*Complete methods.* As stated in the introduction, even small fragments of TRAs are undecidable. A notable exception is the result by Chen and Xia [12], who prove decidability for the univariate fragment with the sine function, where atomic formulas are restricted to rational polynomial inequalities in  $x$  and  $\sin(x)$  (disallowing complex terms inside sine). This is achieved by devising a root isolation algorithm.

## VII. FUTURE WORK

We believe that the MCSAT framework and the infrastructure of our TRA plugin provide a good playground for future research on solving problems in transcendental arithmetic.

Many aspects remain under the implementers' control and warrant further studies. We describe a few of them.

*Degree of polynomial lemmas.* As noted in Section V, our experiments indicate a trade-off between linear lemmas, which scale better, and polynomial lemmas of arbitrary degree, which converge faster. This calls for a systematic study of how the degree of the polynomial lemmas impacts the performance and scalability of our procedure, and for hybrid approaches that tune this degree automatically. Such a study will most likely be specific to each transcendental function.

*Feasibility sets.* We devoted little attention to feasibility sets so far: the TRA plugin currently relies on those computed by the NRA plugin (Section III-D). Incorporating the root isolation algorithm of [12] could yield sharper feasibility sets for the pre-images of trigonometric functions.

*Consistency check.* In the paper and in our prototype, the consistency check is carried out via interval analysis, which is well known in the program analysis literature to yield very crude approximations. This motivated the development of *relational abstract domains* [27], i.e., domains more expressive than intervals that track constraints between variables. We believe that studying these domains in the context of TRAs would improve our consistency check.

Relatedly, the consistency check relies on abstractions solely because the underlying problem of evaluating the sign of a transcendental expression is undecidable for general interval-computable functions. From Richardson's work [28], [29] on checking zeroes of polynomial-exponential expressions over the complex numbers, one derives that the problem is *decidable under Schanuel's conjecture* for literals containing only field operations,  $\sin$ ,  $\pi$ , and  $\exp$ . Correctness holds unconditionally; the conjecture is only needed to guarantee termination. Efficiently integrating this procedure into the consistency check is an interesting direction for future work.

*Satisfiability.* All the improvements described above implicitly aim at building a better solver for *unsatisfiability*: sharper overapproximations can turn  $\delta$ -SAT verdicts into UNSAT ones, but they leave unchanged the class of solutions the solver can return, which remains limited to algebraic solutions. Improving SAT answers instead calls for "repairing"  $\delta$ -SAT verdicts. Promising methods that could be integrated into our tool include those based on topological degree testing [30], strong satisfaction checks [31], and local-search approaches [32], [33], [34].

**Acknowledgments.** This work is part of a project co-funded by the European Union (GA 101154447) and by MCIN/AEI (GA CEX2024-001471-M and GA PID2022-138072OB-I00).



Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or European Commission. Neither the European Union nor the granting authority can be held responsible for them.

The artifact(s) associated with this paper is/are available at Zenodo under the following DOI: 10.5281/zenodo.21705986.

## REFERENCES

- [1] E. Bartocci, J. Deshmukh, A. Donzé, G. Fainekos, O. Maler, D. Ničković, and S. Sankaranarayanan, “Specification-based monitoring of cyber-physical systems: a survey on theory, tools and applications,” in *Lectures on Runtime Verification: Introductory and Advanced Topics*. Springer, 2018.
- [2] S. Ratschan, “Applications of quantified constraint solving over the reals—bibliography,” 2012.
- [3] M. Laczkovich, “The removal of  $\pi$  from some undecidable problems involving elementary functions,” *Proc. Am. Math. Soc.*, 2003.
- [4] A. Macintyre and A. J. Wilkie, “On the decidability of the real exponential field,” in *Kreiseliana. About and Around Georg Kreisel*. A K Peters/CRC Press, 1996.
- [5] B. Dutertre, “Yices 2.2,” in *CAV*, 2014.
- [6] A. Cimatti, A. Griggio, A. Irfan, M. Roveri, and R. Sebastiani, “Incremental linearization for satisfiability and verification modulo nonlinear arithmetic and transcendental functions,” *ACM Trans. Comput. Log.*, 2018.
- [7] G. Kremer, A. Reynolds, C. Barrett, and C. Tinelli, “Cooperating techniques for solving nonlinear real arithmetic in the cvc5 smt solver (system description),” in *IJCAR*, 2022.
- [8] S. Gao, S. Kong, and E. M. Clarke, “dreal: An smt solver for nonlinear theories over the reals,” in *CADE*, 2013.
- [9] G. E. Collins, “Quantifier elimination for real closed fields by cylindrical algebraic decomposition,” in *Quantifier Elimination and Cylindrical Algebraic Decomposition*. Springer, 1998.
- [10] S. Basu, R. Pollack, and M.-F. Roy, *Algorithms in Real Algebraic Geometry*. Springer, 2006, second Edition.
- [11] O. Bournez, “Complexity over the reals,” 2024, available at [www.lix.polytechnique.fr/Labo/Olivier.Bournez/load/MPRI-PROJET/Cours-2024-COMPR.pdf](http://www.lix.polytechnique.fr/Labo/Olivier.Bournez/load/MPRI-PROJET/Cours-2024-COMPR.pdf), accessed August 29, 2026.
- [12] R. Chen and B. Xia, “Deciding first-order formulas involving univariate mixed trigonometric-polynomials,” in *ISSAC*, 2023.
- [13] J. Gallego-Hernández and A. Mansutti, “On the existential theory of the reals enriched with integer powers of a computable number,” in *STACS*, 2025.
- [14] F. Johansson, “Arb: a c library for ball arithmetic,” *ACM Commun. Comput. Algebra*, 2014.
- [15] S. Gao, J. Avigad, and E. M. Clarke, “Delta-decidability over the reals,” in *LICS*, 2012.
- [16] L. De Moura and D. Jovanović, “A model-constructing satisfiability calculus,” in *VMCAI*, 2013.
- [17] D. Jovanovic, C. Barrett, and L. De Moura, “The design and implementation of the model constructing satisfiability calculus,” in *FMCAD*, 2013.
- [18] D. Jovanović and L. De Moura, “Solving non-linear arithmetic,” *ACM Commun. Comput. Algebra*, 2013.
- [19] J. Renegar, “On the computational complexity and geometry of the first-order theory of the reals, part I: introduction. preliminaries. the geometry of semi-algebraic sets. the decision problem for the existential theory of the reals,” *J. Symb. Comput.*, 1992.
- [20] T. Hader, A. Irfan, and S. Graham-Lengrand, “Decision heuristics in mcsat,” in *CAV*, 2025.
- [21] G. A. Baker and P. Graves-Morris, *Padé Approximants*. Cambridge University Press, 2009, second Edition.
- [22] F. Benhamou and L. Granvilliers, “Continuous and interval constraints,” in *Handbook of Constraint Programming*. Elsevier, 2006.
- [23] S. Ratschan, “Efficient solving of quantified inequality constraints over the real numbers,” *ACM Trans. Comput. Log.*, 2006.
- [24] M. Fränzle, C. Herde, T. Teige, S. Ratschan, and T. Schubert, “Efficient solving of large non-linear arithmetic constraint systems with complex boolean structure,” *J. Satisf. Boolean Model. Comput.*, 2007.
- [25] F. Brauße, K. Korovin, M. V. Korovina, and N. T. Müller, “The ksmt calculus is a  $\delta$ -complete decision procedure for non-linear constraints,” in *CADE*, 2021.
- [26] B. Akbarpour and L. C. Paulson, “Metitarski: An automatic theorem prover for real-valued special functions,” *J. Autom. Reason.*, 2010.
- [27] A. Miné, “Weakly relational numerical abstract domains,” Ph.D. dissertation, École Polytechnique, Palaiseau, France, 2004. [Online]. Available: <https://tel.archives-ouvertes.fr/tel-00136630>
- [28] D. Richardson, “How to recognize zero,” *J. Symb. Comput.*, 1997.
- [29] —, “Zero tests for constants in simple scientific computation,” *Math. Comput. Sci.*, 2007.
- [30] E. Lipparini and S. Ratschan, “Satisfiability of non-linear transcendental arithmetic as a certificate search problem,” *J. Autom. Reason.*, 2025.
- [31] A. Eggers, E. Kruglov, S. Kupferschmid, K. Scheibler, T. Teige, and C. Weidenbach, “Superposition modulo non-linear arithmetic,” in *FroCoS*, 2011.
- [32] A. Cimatti, A. Griggio, E. Lipparini, and R. Sebastiani, “Handling polynomial and transcendental functions in smt via unconstrained optimisation and topological degree test,” in *ATVA*, 2022.
- [33] X. Ni, B. Xia, and T. Zhao, “Local search for checking satisfiability of formulas with trigonometric functions,” in *ATVA*, 2025.
- [34] E. Lipparini, T. Hader, A. Irfan, and S. Graham-Lengrand, “Boosting mcsat modulo nonlinear integer arithmetic via local search,” in *CADE*, 2025.